

Trabajo Práctico 2.1

Repaso sobre Expresiones Lambda

1. Dadas las siguientes interfaces:

```
interface A {  
    void metodo();  
}  
  
interface B {  
    void metodo(String b);  
}  
  
interface C {  
    boolean metodo(String c);  
}  
  
interface D<T, R> {  
    R metodo(T c);  
}
```

la siguiente clase:

```
public class AprendiendoLambdas {  
    public void unMetodo(A a) {  
        a.metodo();  
    }  
  
    public void unMetodo(B b) {  
        b.metodo("unString");  
    }  
  
    public void unMetodo(C c) {  
        System.out.println(c.metodo("otroString") ? "true" : "false");  
    }  
  
    public void unMetodo(D<Long, Long> d) {  
        d.metodo(10L);  
    }  
}
```

Y la clase Main:

```
public class Main {
```

```
public static void main(String[] args) {
    AprendiendoLambdas a = new AprendiendoLambdas();
    a.unMetodo((b) → { System.out.println("abcd" + b);});
    a.unMetodo(() → System.out.println("abcd"));
    a.unMetodo((variable) → {System.out.println("abcd");});
    a.unMetodo((variable) → {System.out.println("abcd"); return true;});
    a.unMetodo((Long variable) -> {
        System.out.println("abcd");
        return 10L;
    });
}
```

Indique qué métodos de la clase `AprendiendoLambdas` se invocan en cada caso. Explique claramente por qué.

2. Dada la interfaz:

```
interface C {
    boolean metodo(String c);
}
```

La clase:

```
public class AprendiendoLambdas {
    public void unMetodo(C c) {
        System.out.println(c.metodo("abcd"));
    }
}
```

Escriba una clase `Main` para poder llamar al método `AprendiendoLambdas#unMetodo(C c)` de la siguiente forma:

- Utilizando un lambda dado que imprima `true` si el largo del `String` es par, `false` en caso contrario.
- Utilizando un lambda dado que imprima `true` si el `String` empieza con "a" minúscula, `false` en caso contrario.

3. En el repo:

<https://github.com/enriquemolinari/oop2-lambdas>

Dentro del paquete `ejercicios.filtros` encontrará la clase `Persona` y `Personas`. Utilizando

expresiones lambda refactorice los métodos *nombresQueEmpiezanConE* y *nombresCuyaCantidadDeLetrasEsPar* para quitar el código duplicado.

4. En el repo:

<https://github.com/enriquemolinari/oop2-lambdas>

Dentro del paquete *ejercicios.jdbc* observe la clase *Usuarios*. Refactorícela para sacar la estructura repetida de la gestión de la transacción connection - commit - rollback.

Las clases *Main* y *UsuarioDDL* simplemente están para poder ejecutar el programa.

5. En el repo:

<https://github.com/enriquemolinari/oop2-lambdas>

Dentro del paquete *lambdas.permissions1* observe las clases *Productos* y *SecuritySubSystem*. Refactorice la clase *Productos* utilizando expresiones lambda para remover el código duplicado.

6. En el repo:

<https://github.com/enriquemolinari/oop2-lambdas>

Dentro del paquete *lambdas.permissions2* observe las clases *Productos* y *SecuritySubSystem*. Refactorice la clase *Productos* utilizando expresiones lambda para remover el código duplicado.